

Assignment 15.3

Name:K.Sanjana

Roll.no:2303a52139

Batch no:41

Task 1: Password Strength & Validation Check

Prompt: *Create a basic Flask application. Implement a root endpoint /status that returns a JSON response indicating 'API is live'. Provide the necessary code to run this server locally within the Colab environment and include instructions on how to verify the /status endpoint's response using either a browser or an API testing tool.*

Code:

```
# Generate a Python user registration program check whether Password length is validated
# and whether Weak passwords are allowed.
import re

COMMON_PASSWORDS = ["123456", "password", "admin", "qwerty"]

def is_strong_password(password):
    if len(password) < 8:
        return False, "Password must be at least 8 characters long"

    if password in COMMON_PASSWORDS:
        return False, "Password is too common"

    if not re.search(r"[A-Z]", password):
        return False, "Must contain at least one uppercase letter"

    if not re.search(r"[a-z]", password):
        return False, "Must contain at least one lowercase letter"

    if not re.search(r"[0-9]", password):
        return False, "Must contain at least one digit"

    if not re.search(r"[!@#$%^&*]", password):
        return False, "Must contain at least one special character"

    return True, "Strong password"

username = input("Enter username: ")
password = input("Enter password: ")
```

```
if valid:
    print("User registered successfully!")
else:
    print("Error:", message)
```

Output:

```
C:\Users\Sanjana\Documents\AI Assistant Coding> &
C:\Users\Abhi\Documents\AI Assistant Coding\Assign 20.3.py"
Enter username: Sanjana
Enter password: pass_123me
Error: Must contain at least one uppercase letter

C:\Users\Sanjana\Documents\AI Assistant Coding> &
C:\Users\Abhi\Documents\AI Assistant Coding\Assign 20.3.py"
Enter username: Sanjana
Enter username: Pass_123me
Enter password: Pass_123me
User registered successfully!
```

Explanation:

Enforces minimum entropy through length and complexity

Prevents dictionary attacks via common password rejection

Uses regex-based validation for robust checks

Task 2: Insecure File Upload Handling

Prompt: *Create a Flask application that allows users to upload files. The application should save the uploaded files to a specified directory on the server.*

Code:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    global request_id_counter
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests)

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output:

```
... To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
* Serving Flask app '__main__'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Justification:

Implements basic CRUD (Create + Read)

Uses in-memory storage (simple for lab)

Task 3: Modify Existing Resource Using AI Assistance

Prompt:

Add PUT endpoint to update a request by index with proper error handling in Flask.

Code:

```

from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    global request_id_counter
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests)

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)

```

Output:

```

... To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
* Serving Flask app '__main__'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)

```

Justification:

Enables update operation

Error handling prevents crashes

Task 4: Delete Resource

Prompt: Add DELETE endpoint to remove a request by index with safe error handling.

Code:

```
11 from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    global request_id_counter
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests)

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

@app.route('/requests/<int:request_id>', methods=['DELETE'])
def delete_request(request_id):
    global requests
    original_length = len(requests)
    requests = [req for req in requests if req['id'] != request_id]
    if len(requests) == original_length:
        return jsonify({'message': 'Request with id {request_id} deleted successfully'}), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output:

```
1 v To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
   In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
   * Serving Flask app '__main__'
   * Debug mode: on
   INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
   * Running on all addresses (0.0.0.0)
   * Running on http://127.0.0.1:5000
   * Running on http://172.28.0.12:5000
   INFO:werkzeug:Press CTRL+C to quit
   INFO:werkzeug: * Restarting with watchdog (inotify)
```

Explanation:

Completes create, update, read and delete(CURD) and safe removal with validation.

Task 5: AI-Assisted API Documentation and Readability

Prompt: Add docstrings and comments to Flask API endpoints and optionally integrate Swagger documentation.

Code:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    """
    GET /status
    Returns the API status.

    Responses:
    200: {"status": "ok", "message": "API is running"}
    """
    return jsonify({"status": "ok", "message": "API is running"})

@app.route('/requests', methods=['POST'])
def create_request():
    """
    POST /requests
    Creates a new service request.

    Request Body (JSON):
    {
      "title": "string",
      "priority": "string"
    }

    Responses:
    201: New request object with 'id', 'title', 'priority'
    400: {"error": "Missing title or priority"}

    Global request_id_counter
    data = request.get_json()
    # validate input data
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({"error": "Missing title or priority"}), 400

    # Create a new request object
    new_request = {
        "id": request_id_counter,
        "title": data['title'],
        "priority": data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    """
    GET /requests
    Retrieves all service requests.

    Responses:
    200: A list of all request objects.
    """
    return jsonify(requests)

@app.route('/requests/<request_id>', methods=['PUT'])
def update_request(request_id):
    """
    PUT /requests/<request_id>
    Updates an existing service request by its ID.

    Path Parameters:
    request_id (int): The ID of the request to update.
    """
    # validate input data
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({"error": "Missing title or priority in request body"}), 400

    # Find and update the request
    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({"error": "Request not found"}), 404

@app.route('/requests/<request_id>', methods=['DELETE'])
def delete_request(request_id):
    """
    DELETE /requests/<request_id>
    Deletes a service request by its ID.

    Path Parameters:
    request_id (int): The ID of the request to delete.

    Responses:
    200: {"message": "Request with id (request_id) deleted successfully"}
    400: {"error": "Request not found"}

    Global requests
    original_length = len(requests)
    # Filter out the request to be deleted
    requests = [req for req in requests if req['id'] != request_id]
    if len(requests) == original_length:
        return jsonify({"message": "Request with id (request_id) deleted successfully"}), 200
    return jsonify({"error": "Request not found"}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or None (not specifying)
    # the host might work depending on the Colab environment setup.
    # For a single run, you can just use app.run(debug=True)
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal. If this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output:

```
144 To run the app, execute 'python your_file_name.py' in a terminal. If this were a script.
    In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
    * Serving Flask app "main"
    * Debug mode: on
    INFO:werkzeug WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
    * Running on http://127.0.0.1:5000
    * Running on http://0.0.0.0:5000 (Press Ctrl+C to quit)
    INFO:werkzeug * Restarting with watchdog (inotify)
```

Explanation:

Docstrings → improve readability

Swagger → auto API documentation. Helps other developers integrate faster

